

Compiler Design

Exit Exam Tutorial



Prepared for Year 4 Graduating Class

Prepared by: Bekan Kitaw

Course Name: Compiler Design

Institution: Jimma University, JiT, FCI, Computer Science Departement

Date: April 01, 2025

Test Blueprint for Compiler Design



General Objective	Learning outcome	Cognitive						Total = 6 Questions
		Remember	Understand	Apply	Analyze	Evaluate	Create	
Understand the fundamental principles of compiler design, its various constituent parts, algorithms, and data structures that must be used in the compiler, and provide the skills required for building compilers for various situations encountered in a computer science career.	Specify and analyze the lexical , syntactic and semantic structures of advanced language features			1	2			3
	Describe techniques for lexical and syntax analysis	1						1
	Understand the basic principles of compiler design		1					1
	Describe syntax directed translation and type checking techniques		1					1

Introduction to Programming Languages



- The *evolution of computers* has led to the creation of *hundreds of different programming languages* for various types of development.
- **Programming language** is a *series of commands* that help computers, mobile phones, tablets, and other electronic devices function as intended and perform various tasks.
- There are different types of programming languages, and the correct one must be chosen based on the philosophy and objectives of a particular device or program.
- We can classify these languages into **2** broad categories:
 1. Low-level programming languages
 2. High-level programming languages

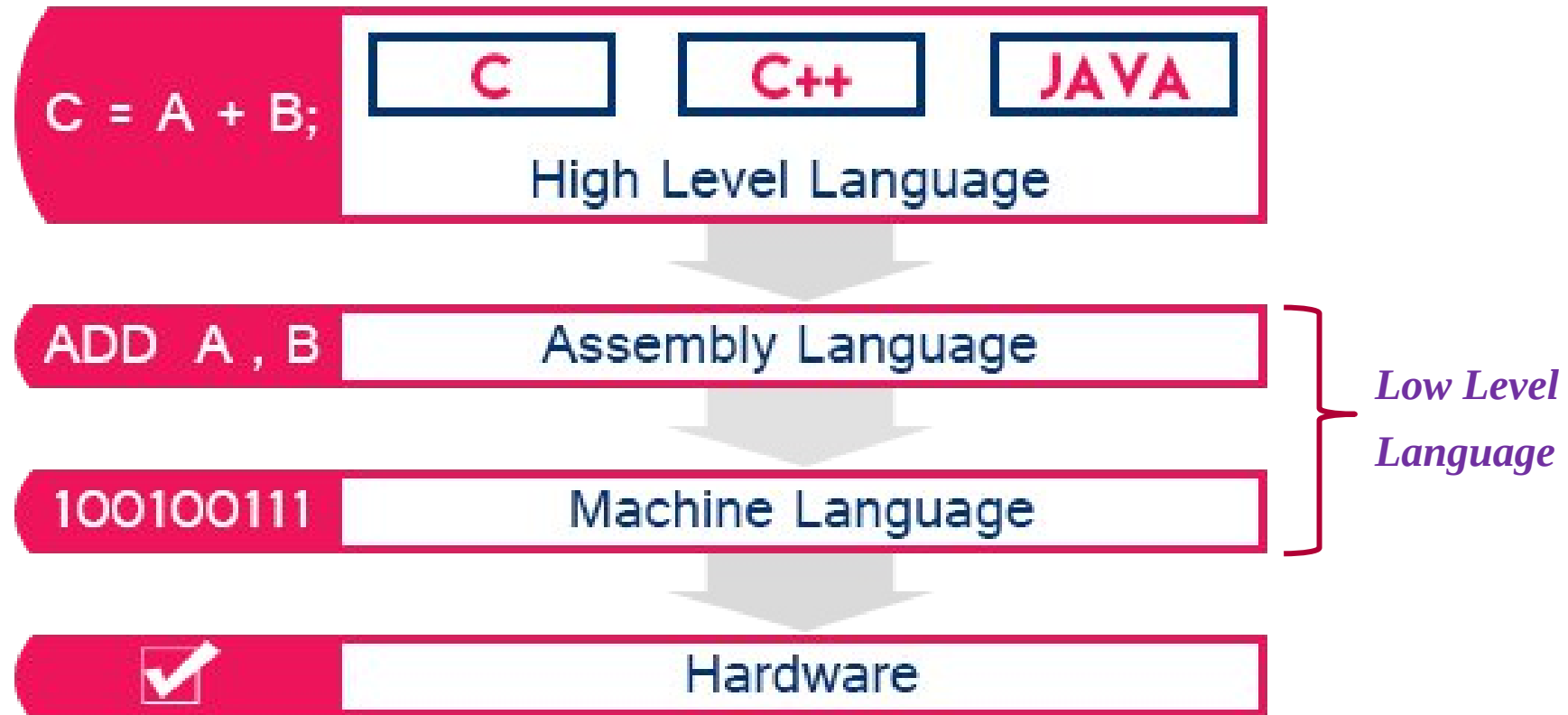


1. **Low-level languages:** Closer to the machine code and require a deeper understanding of computer architecture. **Examples:**
 - **Machine code:** instructions consisting *binary digits (0s and 1s)* & understood directly by the processor.
 - **Assembly language:** Uses *symbolic representations*, making it slightly easier to understand and write than raw machine code. **Examples:** ADD, MOV, XCHG, etc
2. **High-level languages:** uses a *more human-readable syntax*. They contain words and phrases from the languages in common use, English/other languages. **Examples:**
 - *Object oriented programming languages: C++, Java, Python, C#*

Cont'd



- Look at the difference;





- Computer programs are usually written in high level languages.
 - A *high-level programming language* is written in language that is designed to be **easily understood by humans**.
 - It uses syntax that is similar to **human language** to form instructions. **For example** in Python the command “**print**” is used to output data.
- However, computers **cannot** understand high level languages as we humans do. They can only understand the programs that are developed in **binary systems** known as a **machine code**.
- Thus, this **high level languages have to be converted into machine code** so as to be understood by the computer.
- Those programs that are used to convert languages into machine codes are called **language translators**.



- There are different types of programming language translators. These are **compilers**, **interpreters** and **assemblers**.
- Both **compilers** and **interpreters** are programs that are used to convert **high level language** (source code) into the **machine codes**.
- **Compiler** analyze all the language statements that are written in **high level language** to **check if they are correct**.
 - *If it comes across something incorrect, it will give an error message.*
 - *If there are no errors spotted, the compiler will convert the **whole source code into machine code at once**.*
- **Interpreter** also convert the **high level language** into **machine language but line by line**.
- **Assembler** converts assembly language program to machine language.

- For instance, compiler converts....

Java

```
sum = 0;  
for (x = 3; x < 5; x++)  
{ cout << "x is " << x;  
  cout << endl;  
  sum += x;  
  a *= b / 2;  
}
```

a) Source program

Machine Language

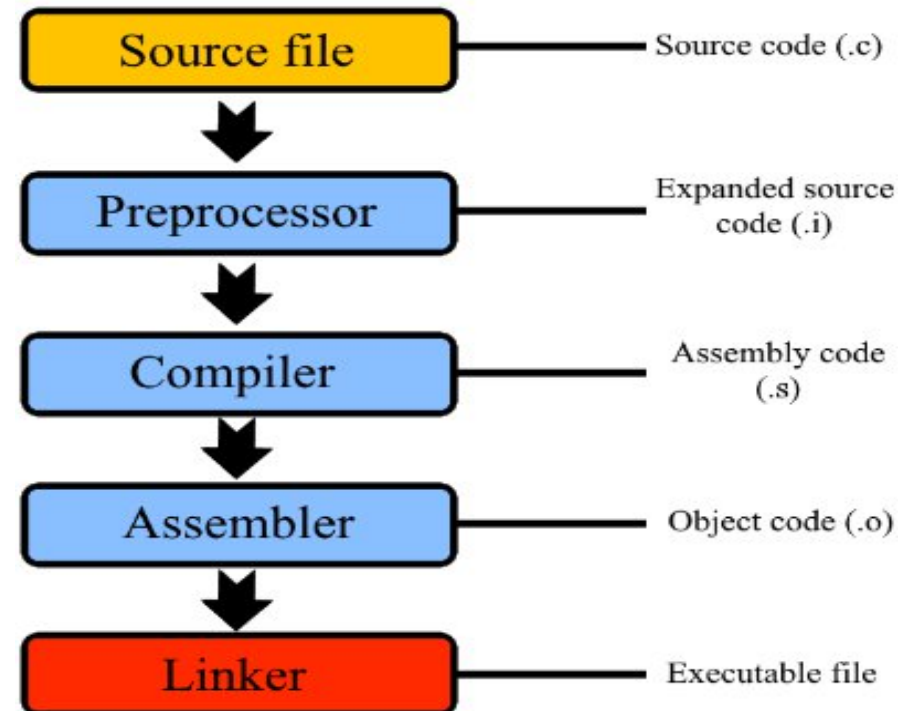
```
10011101  
01011011  
10111100  
01100110  
10101100  
00011011
```

b) Object code(machine code)

Cousins of Compiler



- While a compiler does translating a high-level language to machine code, several other programs can be involved in the process and these are called **compiler cousins**.
- **Cousins of compiler:** software tools that work alongside the compiler but are not directly involved in the core translation process.



Cousins of Compiler(Cont'd)



1. **Preprocessor** : It processes the source code before it reaches the compiler.
 - The preprocessor can perform tasks like:
 - **Conditional compilation**: **Including** or **excluding** parts of the code based on predefined conditions.
 - *Including header files containing function declarations and constant definitions.*
 - *excluding comments from the source program.*
 - **Macro expansion**: Replacing defined keywords with their corresponding code blocks, promoting code reusability.
 - **File inclusion**: Inserting the contents of other files into the source code.
2. **Assembler (if applicable)**: If the compiler translates the code into assembly language then assembler is needed.
 - The assembler translates the assembly language code into machine code specific to the target processor.

Cousins of Compiler(Cont'd)



3. Linker:

- The linker combines multiple object files (compiled code modules) and libraries (pre-written code) into a single executable file.
- ***Example:*** *Imagine Java application with multiple classes. Each class might be compiled into a separate .class file containing its machine code.*
- *The linker then combines these individual files and any necessary libraries into a single executable file (.exe or similar) that the OS can load and run.*

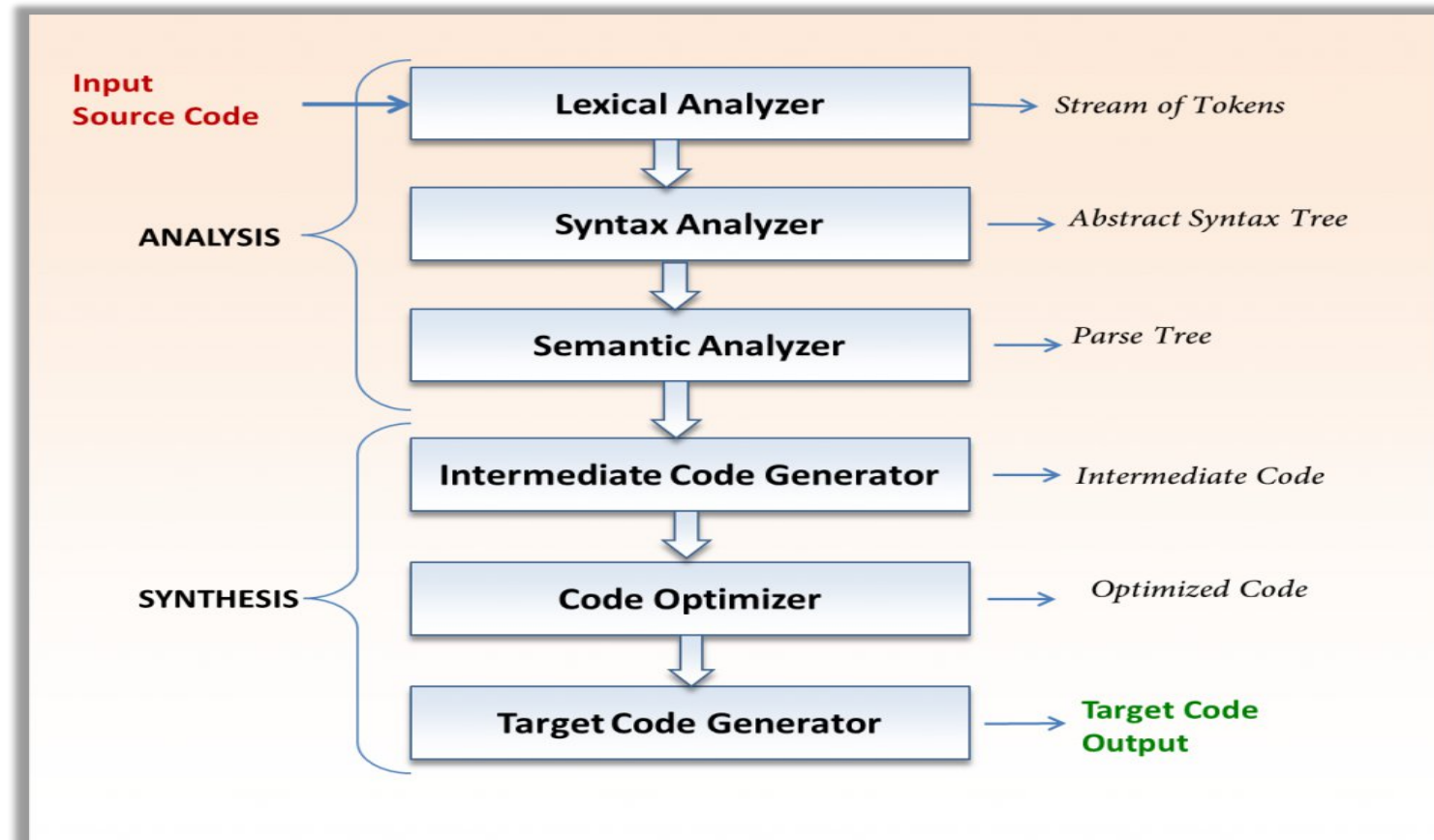
4. Loader:

- The loader takes the executable file created by the linker and loads it into memory for the operating system to execute.
- It prepares the program for execution by allocating memory space and setting up any necessary environment variables.

Phases of Compiler



- Any compiler must perform **two** major tasks each of which has several sub tasks as shown below. I.e. *Analysis* and *Synthesis*





1. **Lexical Analysis (Scanning):** **tokenizer**, breaking down the input program into a stream of meaningful units called **tokens**. These tokens can be **keywords** (like "if" or "for"), **identifiers** (variable names), **operators** (+, -, *), or **punctuation** symbols (semicolons, brackets).
2. **Syntax Analysis (Parsing):** checks if tokens follow the **grammatical rules of the programming language**.
 - It **builds a parse tree** that represents the structure of the program.
3. **Semantic Analysis:** **meaning checks on the program** represented by the parse tree. Verifying things like:
 - **Type compatibility** between expressions and variables.
 - **Usage of variables** before they are declared or defined.
 - **Scope** and accessibility of variables and functions.
4. **Intermediate Code Generation:** translates the program into an **intermediate representation**, which is a code format that is independent of the target machine.



5. **Code Optimization (Optional):** **optimizes the intermediate code** to improve its efficiency and performance. Involves:
 - *Removing unnecessary code, replacing complex operations with simpler ones.*
 - *Reordering code for better execution flow.*
6. **Target Code Generation:** generates the **target machine code**, which the computer's processor(CPU) can directly understand and execute.
 - This code is specific to the target platform (e.g., the specific type of computer processor).
 - *This is like translating the simplified sentence into the language the computer understands (machine code).*

Review Questions



1. Which of these is a low-level programming language?

- A) Python B) Java C) **Assembly language** D) C++

2. What is the role of a compiler?

- A. Execute code line-by-line
B. **Translate entire source code to machine code at once**
C. Convert machine code to high-level code
D. Debug runtime errors

3. Which tool translates assembly language to machine code?

- A. Compiler B. Interpreter C. **Assembler** D. Linker

4. What is the purpose of the semantic analysis phase?

- A. Tokenize the source code
B. **Verify type compatibility and variable scope**
C. Generate machine code
D. Remove comments from code

Review Questions



5. What does a preprocessor do?

- A. Allocates memory for variables
- B. File inclusion and exclusion task**
- C. Executes the final program
- D. Optimizes machine code

6. Which phase of compilation checks for grammatical correctness?

- A. Lexical Analysis
- B. Syntax Analysis (Parsing)**
- C. Semantic Analysis
- D. Code Optimization

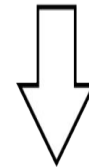
7. What is the output of the lexical analysis phase?

- A. Machine code
- B. Tokens**
- C. Symbol table
- D. Executable file

Chapter 2 - Lexical Analysis

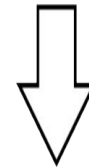
17

i f (x > 3 . 1



Character Stream

**Lexical
Analyzer**



Token Stream

KEYWORD	BRACKET	IDENTIFIER	OPERATOR	NUMBER
"if"	"("	"x"	">"	"3.1"

Introduction to Lexical Analyzer

18

- **Lexical analysis**, also known as **scanning**, is the first stage of compiler that breaks down the source code into smaller, meaningful units called **tokens**. What are tokens?
- **Tokens** are the building blocks of a programming language. They can be:
 - **Keywords**: Words with predefined meanings like **if**, **else**, **for** etc.
 - **Identifiers**: User-defined **names** for variables, functions.
 - **Operators**: Symbols used for calculations like **+**, **-**, *****, **/**
 - **Constants**: Values like **numbers** or **strings**
 - **Punctuation**: Symbols like **commas**, **semicolons**, **parentheses**
- If the lexical analyzer finds a token invalid, it generates **an error**.
- **Lexical Analysis** can be implemented with the ***Deterministic finite Automata(DFA)***.
 - The output is a sequence of **tokens** that are sent to the parser for syntax analysis.

Roles of Lexical Analysis

19

- **Example:** for the expression **int x = 10;**
the lexical analyzer breaks it down into the following tokens:
 - ✓ int (**keyword**)
 - ✓ x (**identifier**)
 - ✓ = (**operator**)
 - ✓ 10 (**literal** or **constant**)
 - ✓ ; (**punctuation**)
- The main responsibilities of LA are therefore;
 - Scans and reads the source code character by character
 - **Tokenization**
 - Eliminates comments and whitespace (**blank**, **newline**, **tab**, and perhaps other characters that are used to separate tokens in the input).
 - Report errors message with line number and display it.
 - Retrieve and update symbol table stores identifier etc...

Tokens, Patterns, and Lexemes

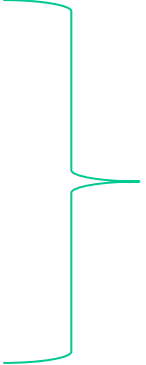
20

- In the world of compiler design, there are three interrelated terms that crucial for understanding how source code is transformed into machine-readable instructions. These are: **tokens**, **lexemes**, and **patterns**
- 1. **Tokens**: are the smallest units of code. They represent individual elements that combine to form a complete program. These are:
 - **Keywords**: Reserved words (e.g., if, else, for, while).
 - **Identifiers**: User-defined names for variables, functions, or classes (e.g., name, calculateArea, MyClass).
 - **Operators**: Symbols used for performing operations (e.g., +, -, *, /, ==).
 - **Constants**: Fixed values like numbers or strings (e.g., 10, "Hello World").
 - **Punctuation**: Special characters (e.g., :, (,), ,).

2. **Lexemes:** these are the actual sequences of characters that make up a specific token.

- They are the concrete representations of a token in the source code.
- While a token defines the category (e.g., keyword, identifier), the **lexeme is the physical string that forms that token instance**.
- **Example:** in the following code

```
int a = 5;
```

- int,
 - a,
 - =,
 - 5,
 - ;
- 
- Lexemes**

3. **Patterns:** these are the **rules or guidelines used to identify lexemes** in the source code.
- These patterns are often expressed using **regular expressions**, which are powerful tools for matching specific character sequences.
 - The lexical analyser (or scanner) in a compiler utilizes these patterns to break down the source code **character-by-character** and **group them into meaningful tokens**.
 - Think of patterns as instructions for the scanner, like "*look for any sequence of letters and digits to identify an identifier*" or "*match the characters + or - to recognize addition or subtraction operators*."

Example: int MAX(int a, int b)

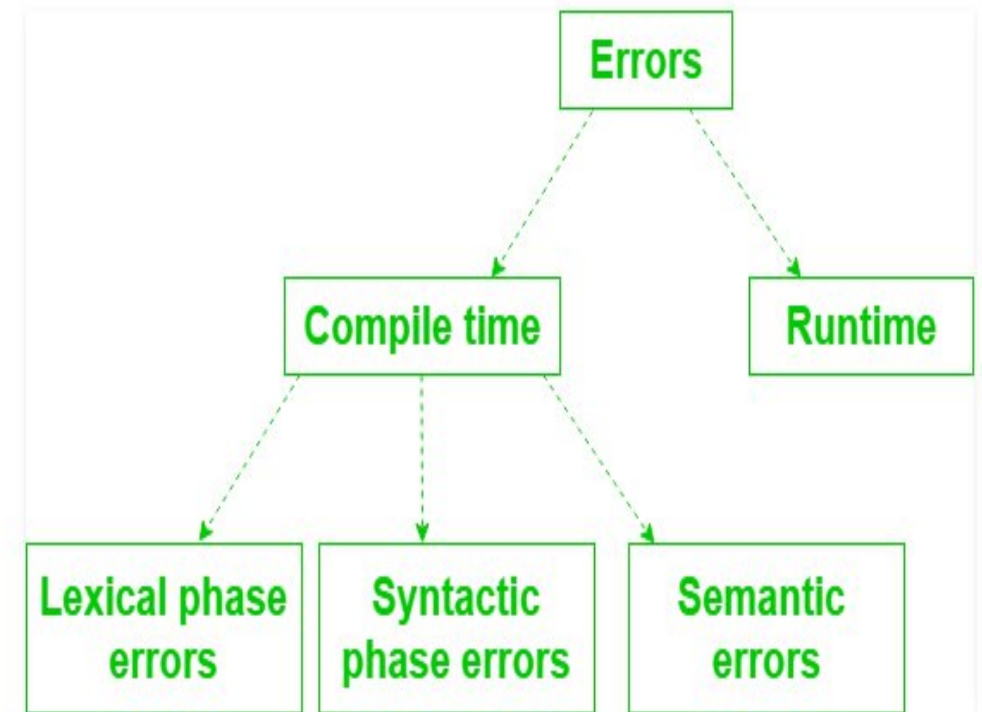
Patterns use RE
to identify this
lexemes!

Lexeme	Token
int	keywords
MAX	identifier
(	operator
int	keyword
a	identifier
,	operator
int	keyword
b	identifier
)	operator

Error detection and Recovery in Compiler



- **Error detection and recovery** helps find and fix mistakes in code during **compilation**.
- The compiler checks for errors and shows messages to the user.
- These errors can be **compile-time** (during translation) or **run-time** (during execution).



Lexical phase errors



- Detected during the **lexical analysis** phase and involves **violation** of the language's **tokenization rules**. Typical lexical errors includes;
 - ❑ **Exceeding length of identifier or numeric constants.**
 - `int veryLongVariableNameThatExceedsLimit = 10; // Error: exceeding maximum length`
 - ❑ **Appearance of illegal characters**
 - `cout<<"Well come"; $ // Error: $ appears at the end of statement.`
 - ❑ **Unmatched or missing string**
 - `Cout<<"Well come; // Error: Unmatched string, missing closing quote.`
 - ❑ **Invalid identifier**
 - `int 1stNumber = 10; // Error: Identifier cannot start with a number`
 - ❑ **Having a space in the middle of a variable name**
 - `Char a b; // Error: space at the middle`

Approaches of building Lexical Analyzer

26

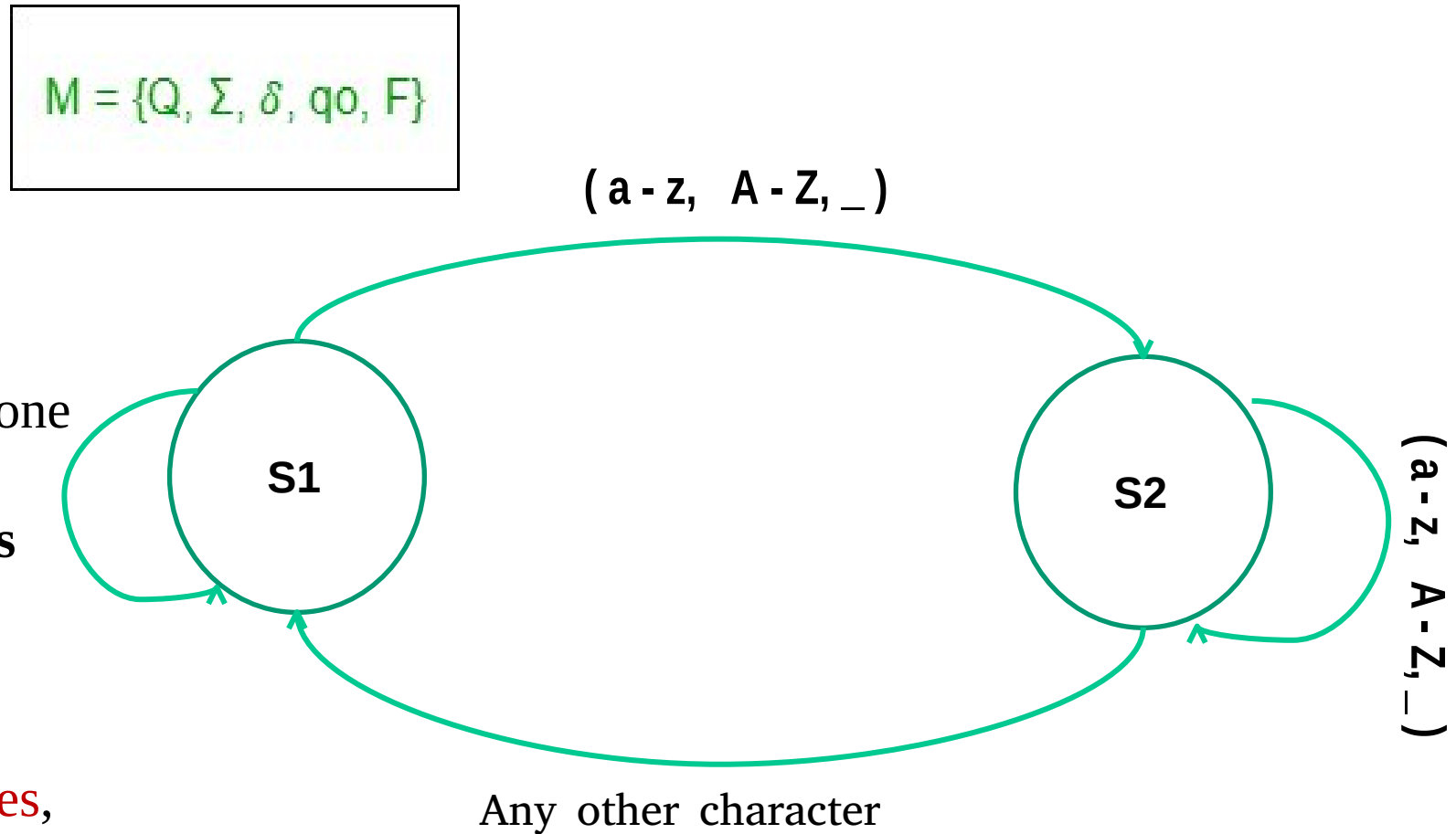
- **Lexical analyser** (also called a **scanner**) is the **first stage** that breaks down the source code into meaningful units called **tokens**.
- There are several approaches of building Lexical Analyser(different techniques used to create this scanner program);
 1. Finite Automata (FA)
 2. Regular Expressions (RE)

1. Finite Automata (FA)

27

- An **Automaton** is an abstract machine that can be in different states.
 - It can change between these states based on rules or inputs.
 - Think of a **light switch**: it can be **ON** or **OFF** (**States**).
 - You **flip the switch** (**the rule**) to change the state.
 - A **Finite Automata (FA)**, also known as a **Finite State Machine (FSM)**, is a classic example where:
 - ✓ The machine is modeled with **limited states**.
 - ✓ It **transitions** between states **based on input characters**.
- ? The **goal** is to determine if the **input sequence** "**matches**" a **specific pattern** the FA is designed for.

- In general a finite automaton consists of **5 tuples** :
 - a **finite set** of states
 - a **start state**
 - a set of **final** (or accepting) states
 - a **set of transitions** T from one state to another.
 - a finite set of **input symbols** (the **alphabet**).
- We can represent a FA graphically, with **nodes** for **states**, and **arcs** for **transitions**.



Lexical Analysis with FA

29

- Let's break down how a Finite Automata (FA) can be used for lexical analysis in the Java program

int x = 5;

- As we have seen previously, the tokens in the given statement are:

int : **Keyword** denoting data type (integer)

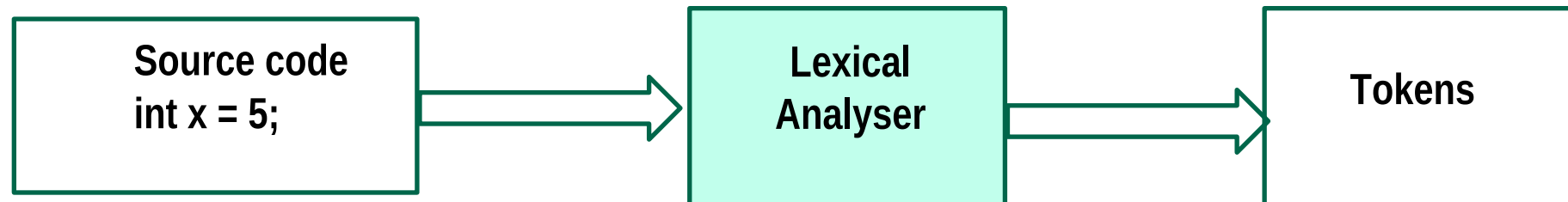
x : **Identifier** (variable name)

= : **Assignment** operator

5 : **Integer** literal (numeric value)

; : **Semicolon** (statement terminator)

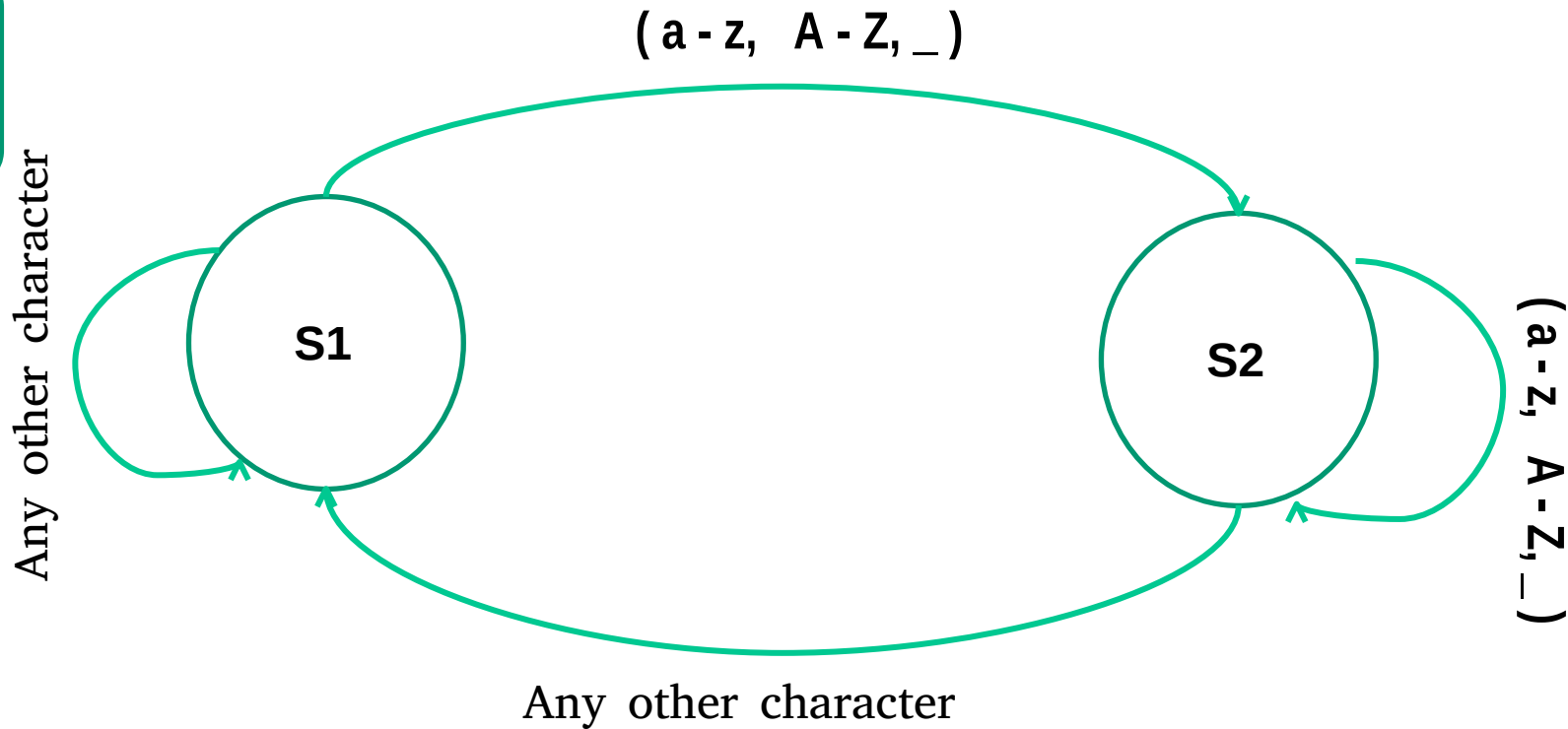
- Lexical Analyser Process is:



- Look at the following representation(recognizing **identifiers** using FA):

In this example,

- S1 = start state
- S2 = is final state



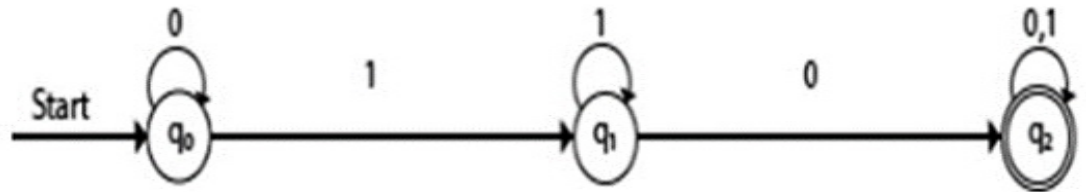
Deterministic Finite automata - DFA

31

- **DFA** is a type of **finite automaton** where, **for each input symbol, there is exactly one corresponding transition**, meaning there is a single resultant state.
- It is a set of five tuples represented as,

$$M = \{Q, \Sigma, \delta, q_0, F\}$$

- Where,
 - Q : A non-empty finite set of states in finite control(q_0, q_1 , etc)
 - Σ : A non-empty finite set of input symbols.
 - δ : It is a transition function that takes two arguments, a state, and an input symbol, it returns a single state.
 - q_0 : It is starting state, one of the states in Q .
 - F : It is a non-empty set of final states/ accepting states from the set belonging to Q .



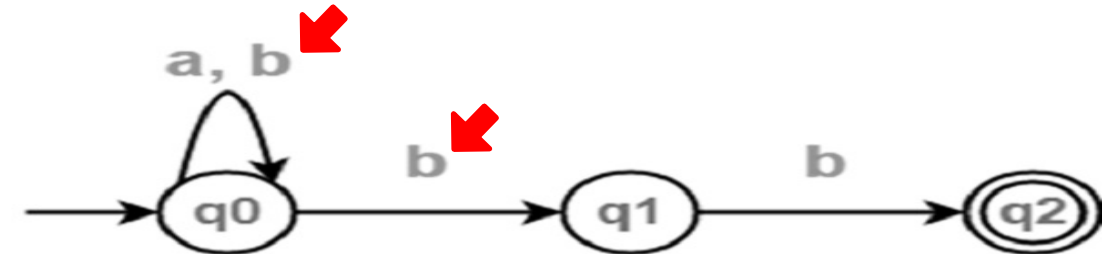
Non - deterministic Finite automata - NDFA

32

- A Finite Automata(FA) is said to be non-deterministic if **there is more than one possible transition** from one state on the same input symbol.
- A non-deterministic finite automata is also a set of five tuples and represented as,

$$M = \{Q, \Sigma, \delta, q_0, F\}$$

- Where,
 - Q : A set of non empty finite states.
 - Σ : A set of non empty finite input symbols.
 - δ : It is a transition function that takes a state from Q and an input symbol from Σ and returns a subset of Q .
 - q_0 : Initial state of NFA and member of Q .
 - F : A non-empty set of final states and member of Q .



2. Regular Expression(RE)

33

- **Regular Expressions (REs)** are a powerful tool for building lexical analysers.
- They are used to define patterns for matching specific sequences of characters.
- Imagine a concise way to represent patterns instead of listing every possible character sequence.
- REs provide a compact notation for defining these **patterns**.
- **How do they work?**
 - REs use a combination of special characters and literal characters to represent patterns.
 - These special characters have specific meanings, such as matching any single character (.), matching a range of characters ([a-z]), or repetition (*, +).

Regular Expression – Specifying token



- **Example:** Suppose we wanted to describe the set of **valid identifiers** so a RE will generate a pattern such as:
 - **Letter(letter | digit)***
 - **Note:** RE uses the **pipe** (|) symbol to match any of the keywords(strings) like "if", "else", or "for".
- RE involves a combination of strings of symbols from some alphabet Σ , **parentheses**, and the **operators** **+** (union), **▪** (concatenation) and ***** (kleene star).

Building Regular Expressions



- Assume that $\Sigma = \{a, b\}$
 - **Zero or more:** a^* means “zero or more a ’s”, To say “zero or more a ’s,”
 - **Example:** to represent $\{\epsilon, ab, abab \dots\}$ you need to say $(ab)^*$.
 - **One or more:** a^+ means “one or more a ’s”,
 - **Example:** "1a", "12a3a", "abc4aa56" (but not the string "bc12e").
 - you can use aa^* (or equivalently a^*a) to mean “one or more a ’s”.
 - **Zero or one:** $(a)?$ It can also be described as an optional ‘ a ’ with $(a + \epsilon)$.
 - **Example:** "1a", "12", "bc4a56" (but not the string "aabc12e").



Example: REs for a Simple Language

- Let's consider a simple language with the following tokens:
 - **Keywords:** if, else, for
 - **Identifiers:** Start with a letter, followed by letters, digits, or underscores (e.g., x, name1, temp_2)
 - **Integers:** one or more sequences of digits, (e.g., 10, 123, 456)
 - **Semicolon (;)**
- Here's how REs can be used to identify these tokens:

1. Keywords:

RE: (if|else|for) - This RE uses the **pipe (|)** symbol to match any of the three keywords "if", "else", or "for".



2. Identifiers:

RE: `[a-zA-Z][a-zA-Z0-9_]*` - This RE defines an identifier pattern:

- `[a-zA-Z]`: Matches the first character to be a letter (a-z or A-Z).
- `[a-zA-Z0-9_]*`: matches zero or more occurrences of letters, digits (0-9), or underscores (_).

3. Integers:

RE: `[0-9]+` - This RE matches one or more digits (0-9), representing an integer literal.

4. Semicolon:

RE: `;` - This simply matches the literal semicolon character.

Tokenization Process with REs

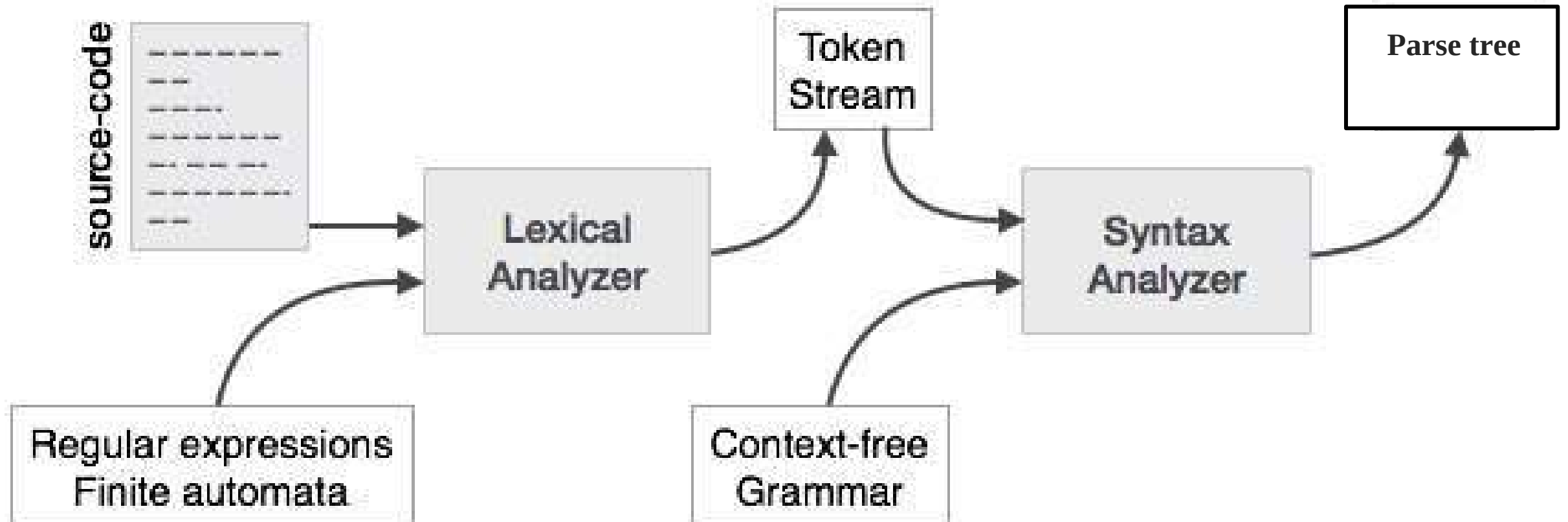


Example: `if x = 10;` then the RE will identify the token using the previous rule(language)

- The lexical analyzer uses the RE defined on previous slide to identify token as follows:
 - Match the first three characters (`if`) with the keyword RE, identifying "if" as a **keyword** token.
 - Match the next character () with **whitespace** (not part of any RE), discarding it.
 - Match the identifier RE with `x`, identifying "x" as an **identifier** token.
 - Match the space () again, **whitespace**, hence discarded.
 - Match the equal sign (`=`) as a separate token (**not defined by an RE in this example**).
 - Match the integer RE with `10`, identifying "10" as an **integer** literal token.
 - Match the semicolon RE with `;`, identifying ";" as a **semicolon** token.

Chapter - 03 Syntax Analyser

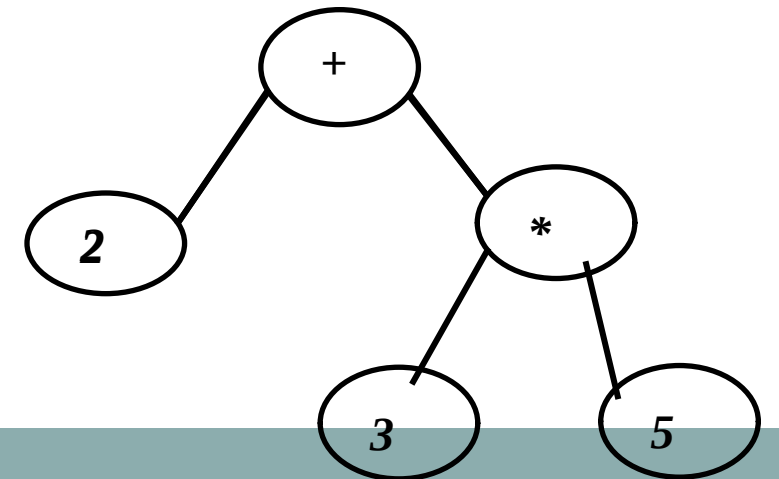
39



Introduction to Syntax Analysis

40

- **Syntax analysis**, also known as **parser**, is the second phase in compiler.
- It checks the structure of the source code to ensure it follows the language's **grammar rules** by generating a **parse tree**.
 - Hence, the output of syntax analysis phase is a **parse tree** or **syntax tree**.
- **For example**, let's consider: "2 + 3 * 5".
 - Syntax analyzer will create a parse tree that demonstrates the **correct order of operations**, such as **multiplication before addition**, to ensure the expression is syntactically correct.
- In this parse tree, the **+** operation is at the top node, with "2" on the left and the * operation on the right.
- The ***** operation has "3" on the left and "5" on the right, indicating that * should be performed before + in this expression.



- A **parser** is a program that takes **tokens** and **grammars** (CFGs) as input and validates them against the grammar.
- **Context-free grammar (CFG)** describes the syntax of a programming language.
- The **parser** checks whether a source program follows the rules defined by the **CFG**.
 - If valid, the parser generates a **parse tree**.
 - If not, the parser provides **error messages**.

Context free grammars, CFG



- Learning a new language, like **English**, involves understanding basic sentence structure.
- Similarly, in programming, **context-free grammars (CFGs)** are used for **syntax analysis**.
- **CFGs** act as a guide, specifying how different parts of your code (such as **keywords**, **variables**, and **operators**) can be arranged to form valid code.
 - They define the **rules for structuring code**, ensuring it follows the correct syntax.
- A CFG can be represented as a **4-tuple**: $\{S, T, N, P\}$, where:
 - ✓ **T** is a finite set of terminal symbols (**tokens**).
 - ✓ **N** is a finite set of non-terminal symbols.
 - ✓ **S** is the start symbol (a non-terminal symbol).
 - ✓ **P** is a finite set of production rules.

$$\text{CFG} = \{S, T, N, P\}$$

Components of CFG



- **Terminal Symbols:** are the **basic building blocks** of our code, similar to words in a sentence. They include **keywords** (if, while), **identifiers** (variable names), **operators** (+, -, *, /), **punctuation** (;, ., (), {}), and **literals** (numbers, strings).
- **Non-Terminal Symbols:** represent **statement**, **expression**, **block**, or **function_declaration**.
- **Production Rules:** These rules **define how non-terminal symbols can be built** using terminal and non-terminal symbols. They're like instructions saying, "A statement can be made with an identifier, followed by an equal sign (=), then an expression, and finally a semicolon (;)."
 - *Production rules are written with an **arrow** (->) separating the non-terminal on the left from its possible combinations on the right.*
- A **start symbol**, is a **special nonterminal** symbol, that **appears in the initial string** generated by the grammar.



- **Example2:** Imagine a CFG rule for assigning a value to a variable
assignment_stmt -> identifier = expression ;
 - **Non-Terminal Symbol:** *assignment_stmt* represents the concept of an assignment statement.
 - **Terminal Symbols:** *identifier* (variable name), *=* (equal sign), *expression* (could be a number, variable, or calculation), and *;*(semicolon).
 - **Production Rule:** This rule says a *valid assignment statement* (*assignment_stmt*) can be formed by an *identifier*, followed by an *=*, then an *expression*, and a semicolon.
 - **Start symbol:** *assignment_stmt* is the only start symbol for this specific example.

Production

<i>expression</i>	$\rightarrow$	<i>expression</i> + <i>term</i>
<i>expression</i>		<i>expression</i> - <i>term</i>
<i>expression</i>	$\rightarrow$	<i>term</i>
<i>term</i>	$\rightarrow$	<i>term</i> * <i>factor</i>
<i>term</i>	$\rightarrow$	<i>term</i> / <i>factor</i>
<i>term</i>	$\rightarrow$	<i>factor</i>
<i>factor</i>		(<i>expression</i>)
<i>factor</i>	$\rightarrow$	id

- In this grammar;
 - The **terminal symbols** are: **id + - * / ()**
 - The **non terminal symbols** are *expression*, *term* and *factor*.
 - The **start symbol** is ***expression***.

Derivations and the CFG



- A **derivation** is a sequence of rewrites that transform the start symbol, S , of a CFG into a string of terminals using the grammar rules (productions).
- It shows step-by-step how a valid code structure can be generated from the start symbol, S .

Example: Derivation of “ab” (using the CFG given below):

Start with S :

Apply $S \rightarrow AB$:

Rewrite A (using $A \rightarrow a$):

Rewrite B (using $B \rightarrow b$):

CFG: $S \rightarrow AB$

$A \rightarrow aAA \mid aA \mid a$

$B \rightarrow bB \mid b$

Non terminal = $\{S, A, B\}$.

Terminal = $\{a, b\}$

Production rules: all rule

Start symbol: S

- This derivation shows that the string "ab" can be generated from the start symbol S by following the grammar rules.
- Furthermore, this derivation can be classified as **LMD** and **RMD**.

Leftmost Derivation(LMD)



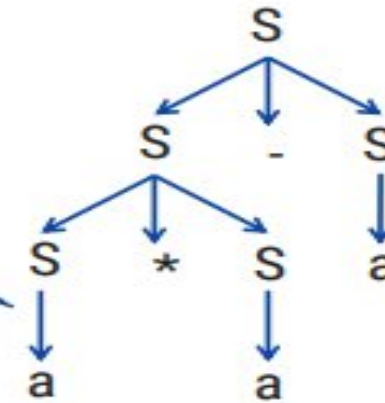
- A derivation of a string W in a grammar G is a **left most derivation** if at every step the left most non terminal is replaced.

Grammar: $S \rightarrow S+S \mid S-S \mid S*S \mid S/S \mid a$ **Output string:** $a * a - a$

S
 $\rightarrow \underline{S}-S$
 $\rightarrow \underline{S}*S-S$
 $\rightarrow a*\underline{S}-S$
 $\rightarrow a*a-\underline{S}$
 $\rightarrow a*a-a$

Leftmost Derivation

Parse tree represents the structure of derivation



Parse tree

➤ Can you find any other LMD for this string on a given CFG?

Rightmost Derivation(RMD)

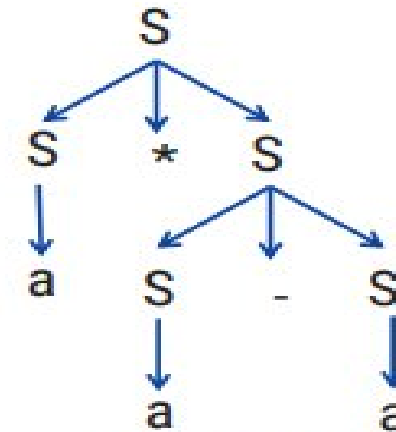


- A derivation of a string W in a grammar G is a **rightmost derivation** if *at every step the right most non terminal is replaced*. It is also called **canonical derivation**.

Grammar: $S \rightarrow S+S \mid S-S \mid S*S \mid S/S \mid a$ **Output string:** $a * a - a$

S
 $\rightarrow S*\underline{S}$
 $\rightarrow S*\underline{S}-\underline{S}$
 $\rightarrow S*\underline{S}-a$
 $\rightarrow \underline{S}*a-a$
 $\rightarrow a*a-a$

Rightmost Derivation



Parse Tree

Example #2 - Leftmost, rightmost Derivation



- Consider the grammar $S \rightarrow S+S \mid S*S \mid a \mid b$. Find leftmost and rightmost derivation for the string $w = a*a+b$.
- Solution:**

Leftmost derivation

$$\begin{aligned} S &\xRightarrow{L} S * S \\ &\xRightarrow{L} a * S \\ &\xRightarrow{L} a * S + S \\ &\xRightarrow{L} a * a + S \\ &\xRightarrow{L} a * a + b \end{aligned}$$

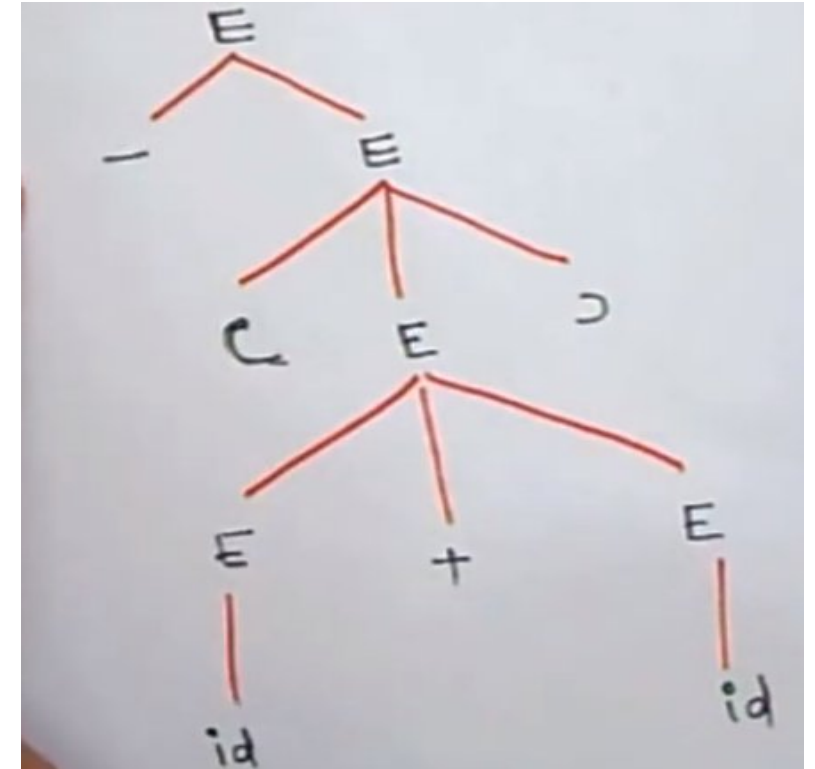
Rightmost derivation

$$\begin{aligned} S &\xRightarrow{R} S * S \\ &\xRightarrow{R} S * S + S \\ &\xRightarrow{R} S * S + b \\ &\xRightarrow{R} S * a + b \\ &\xRightarrow{R} a * a + b \end{aligned}$$

Derivation Tree



- A **derivation tree** is a graphical representation of derivation that filters out the order of replacing non-terminals.
 - *Root node* = start symbol
 - *Interior nodes* = non-terminals
 - *Leaves* = terminals.
- **Example 1:** look at the ff derivation;
 - CFG: $E \rightarrow E + E \mid E * E \mid -E \mid (E) \mid id$
 - Input string: $-(id + id)$

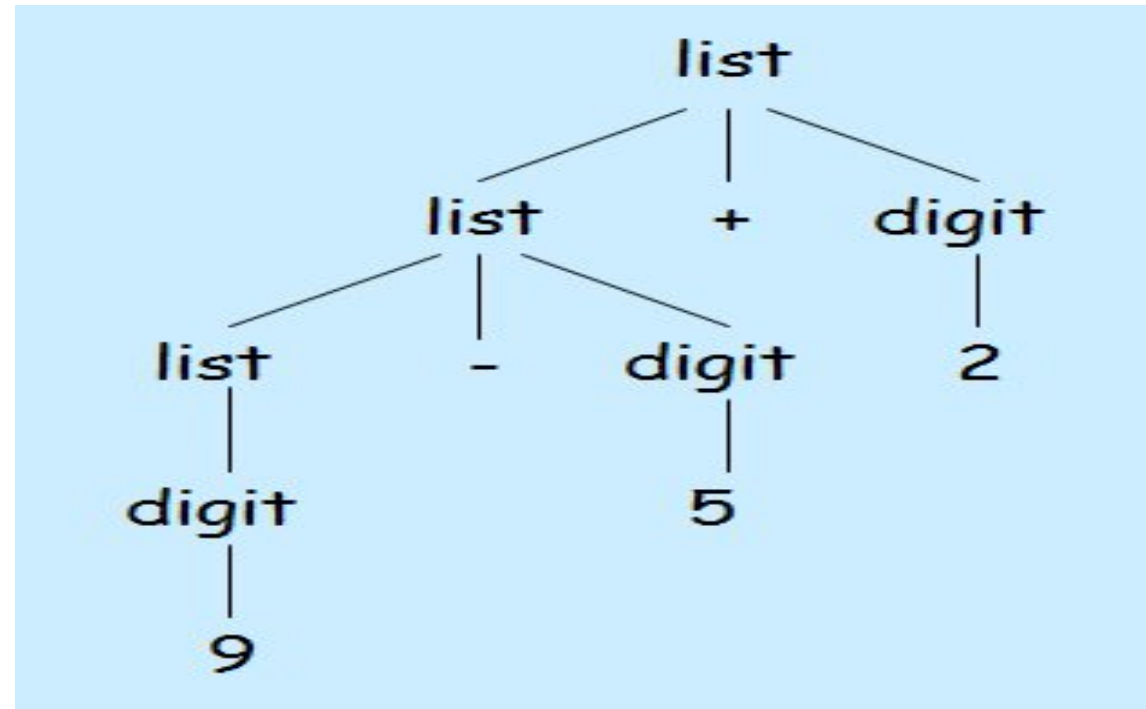




- **Example 3:** Construct parse tree for the given grammar:

G: $\text{list} \rightarrow \text{list} + \text{list} \mid \text{list} - \text{digit} \mid \text{digit}$ Input string: $9 - 5 + 2$

- Parse tree:





- **P:** Define how non-terminals can be rewritten using terminals and other non-terminals. These are the grammar rules that specify valid string structures. Here are the six productions (P):
 - $S \rightarrow AB$ (*The start symbol S can be rewritten as the concatenation of A and B*)
 - $A \rightarrow aAA$ (*A can be rewritten as "a" followed by two "AA"s*)
 - $A \rightarrow aA$ (*A can be rewritten as "a" followed by another A*)
 - $A \rightarrow a$ (*A can be rewritten as a single "a"*)
 - $B \rightarrow bB$ (*B can be rewritten as "b" followed by another B*)
 - $B \rightarrow b$ (*B can be rewritten as a single "b"*)



CFG: $S \rightarrow AB$

$A \rightarrow aAA \mid aA \mid a$

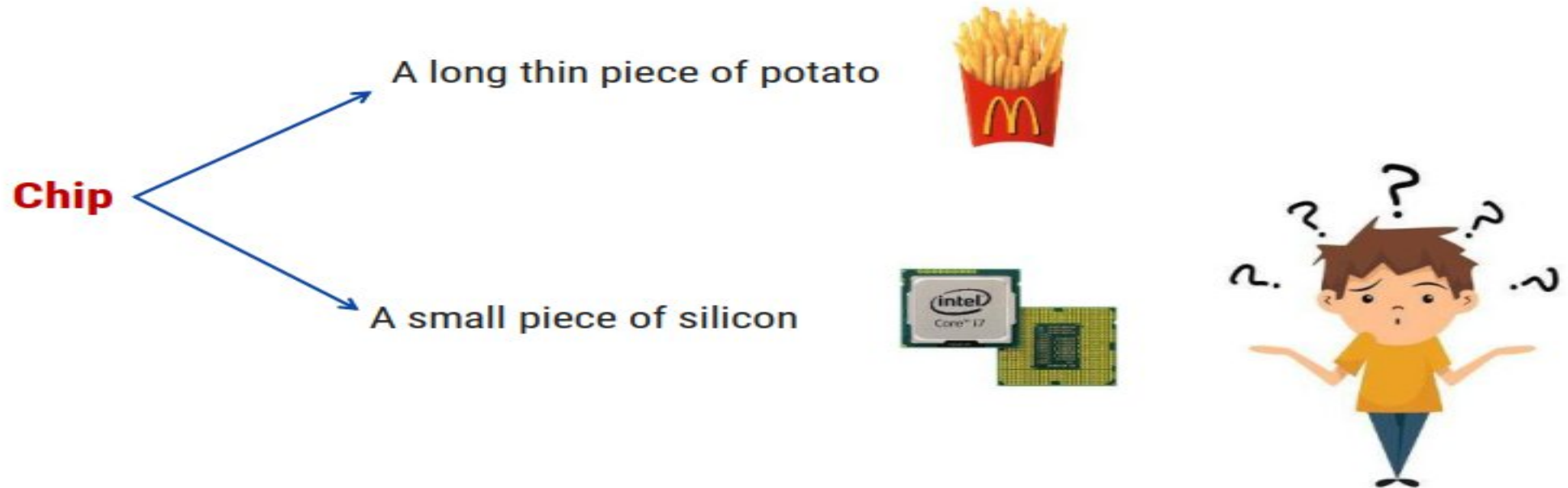
$B \rightarrow bB \mid b$

- **Q2.** We can use the CFG to determine whether the given input strings (ab, aab, aaab, aabba) are valid according to the grammar rules. Here's the analysis for each string:
 - **ab:** We can derive "ab" using the productions for S. **Accepted.**
 - **aab:** A valid derivation exists: $S \rightarrow AB \rightarrow aA$ (using $A \rightarrow a$) $B \rightarrow aab$ (using $B \rightarrow b$). **Accepted.**
 - **aaab:** We can derive "aaab" using the productions for S. **Accepted.**
 - **aabba:** We can't derive "aabba" using the productions for S because they don't allow for two consecutive "b"s within the "B" non-terminal. **Not accepted.**

Ambiguity and Eliminating ambiguity



- In English, ambiguity refers to the state of having **multiple possible meanings** or interpretations. Something ambiguous can be understood in more than one way, leading to confusion or uncertainty.



- In context-free grammars (CFGs), **ambiguity** arises when a single string can have **multiple valid derivation paths** or **parse trees**. This can create confusion for parsers (programs that analyse code structure) and lead to unexpected interpretations of the code.
- **Example 1:** Consider the CFG $S \rightarrow S + S \mid S * S \mid a \mid b$ and string $w = a * a + b$, show the leftmost derivations.

First leftmost derivation

$$\begin{aligned} S &\Rightarrow S * S \\ &\Rightarrow a * S \\ &\Rightarrow a * S + S \\ &\Rightarrow a * a + S \\ &\Rightarrow a * a + b \end{aligned}$$

Second leftmost derivation

$$\begin{aligned} S &\Rightarrow S + S \\ &\Rightarrow S * S + S \\ &\Rightarrow a * S + S \\ &\Rightarrow a * a + S \\ &\Rightarrow a * a + b \end{aligned}$$

Recursion in Syntax Analysis



- **Recursion** is a programming technique where a **function calls itself** within its definition. Here's a breakdown of different types of recursion and how they relate to syntax analysis:
 1. **General Recursion:** This is the most general form of recursion, where the function call to itself can **happen anywhere within the function body**.
 - This occurs when a non-terminal appears on the right-hand side of its own production rule, but not necessarily as the first symbol or the last symbol. **Example:**
$$\text{Expr} \rightarrow \text{Expr} + \text{Term}$$
$$\text{Expr} \rightarrow \text{Term}$$
$$\text{Term} \rightarrow \text{Id}$$
- Here, **Expr** is defined **recursively**. It can either be another Expr followed by a Term (using the "+"), or it can simply be a Term itself. This allows for building complex expressions with multiple operators and operands.



2. Left Recursion:

- This specific type of recursion occurs when the first symbol on the right-hand side (RHS) of a production rule (defining a non-terminal) is the same as non-terminal itself.
- That is, a left recursion happens when a non-terminal symbol derive a string that starts with itself.
- **Example:**
$$E \rightarrow E + b \mid a \quad (\text{Left Recursion})$$
- This CFG has left recursion because E is defined on the RHS as starting with E itself. This type of recursion can lead to infinite loops during parsing and is generally avoided in CFGs for practical reasons.



3. Right Recursion:

- This type of recursion occurs when the **last symbol** on the RHS of a production rule is the same as non-terminal itself.
- That is, a right recursion happens when a non-terminal symbol derive a string that **ends with itself**.
- **Example:**
$$\text{Expr} \rightarrow \text{Term} + \text{Expr} \quad // \text{ Expression can be a term followed by expression}$$
$$\text{Term} \rightarrow a \quad // \text{ Term is a single variable}$$
- Here, the non-terminal Expr appears as the last symbol on the right-hand side of its own production.



Parsing Techniques

Parsing Techniques



- **Parsing** is a process that constructs a syntactic structure (i.e. parse tree) from the **stream of tokens**.
- We categorize these parsing approaches into **two types** based on how the parsing process is carried out;
 1. **Top - Down (LL Parser)** - the parse tree is created top to bottom,
 - starting from the root.
 2. **Bottom - Up (LR Parser)** - the parse is created bottom to top;
 - starting from the leaves.
- Both **top-down and bottom-up parsers** scan the input from **left to right** (one symbol at a time).

1. Top Down Parsing (LL Parser)



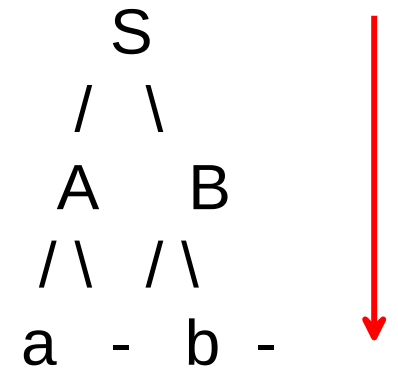
- A **Top-down parser** create parse tree from **root** towards the **leafs** scanning input from **left to right** generating **leftmost derivation** of the input string.
- It is also known as an **LL parser**, where "L" stands for **left-to-right scanning** of the input and the second "L" represents the construction of a **leftmost derivation** of the input string.

- **Example:** Consider the grammar $S \rightarrow AB \mid A \rightarrow a \mid B \rightarrow b$

Assume Input: “ab”.

Using **Top-down approach** starts with S and recursively expands it:

- First, S is expanded to AB.
- Then, A is expanded to a and B is expanded to b.



- It may require **backtracking** (**Recursive-Descent Parsing**), some others are **backtrack-free** (**Predictive parsing**) .

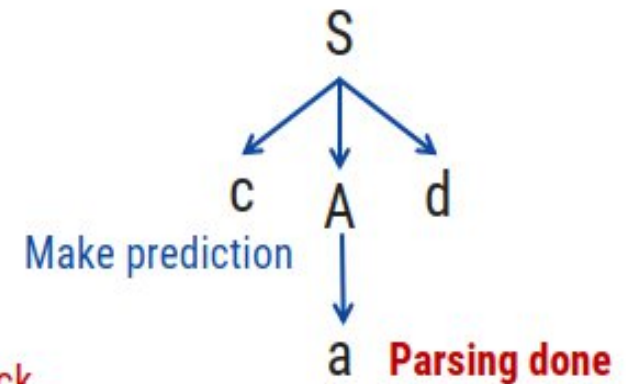
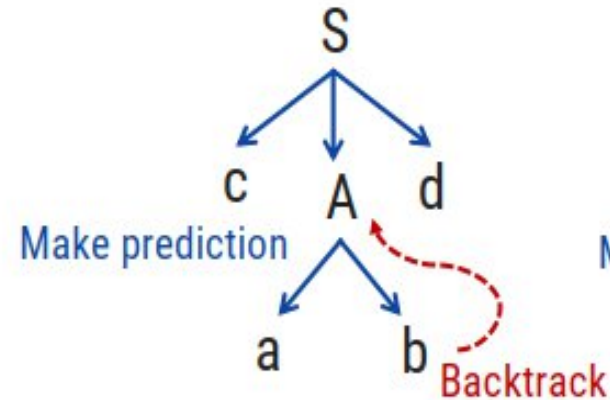
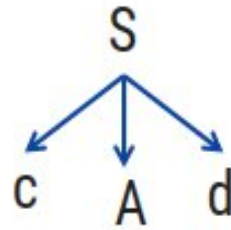
1.1 Recursive descent parsing: Example #1



- Consider the grammar with input string “**cad**”:

$S \rightarrow cAd$

$A \rightarrow ab \mid a$



- Backtracking is needed.
- It tries to find the left-most derivation

1.2 Predictive Parsing (LL Parser)



- **Definition:** A type of top-down parser that uses a **lookahead token** to decide which rule to apply. It can parse an input string deterministically (i.e., no backtracking).
- **How it works:** The "LL" stands for **Left-to-right scan of the input**, and **Leftmost derivation**. It uses a predictive table to decide which rule to apply based on the current input token.
- No backtracking is needed.

ii. Bottom-up parsing(LR)

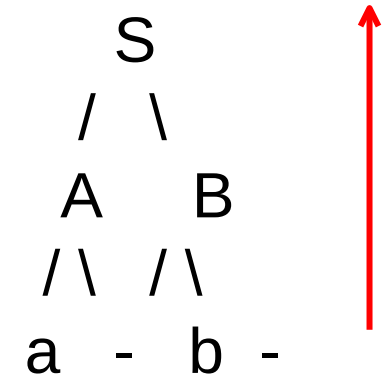


- **Constructs parse tree** for an input string beginning at the **leaves (the bottom)** and working towards the **root (the top)**.
- Reduces a sequence of tokens to the **start symbol**.
- It is also known as **LR parsers** where 'L' stands for it read input from **Left to right** and 'R' stands as it construct a **Rightmost derivation** of the input string.
- **Example:** Consider the grammar: $S \rightarrow AB \mid A \rightarrow a \mid B \rightarrow b$

Assume Input: “ab”.

Using **Bottom-up approach** the parser

- Starts with ab and reduce a to A ($A \rightarrow a$).
- Then reduce b to B ($B \rightarrow b$).
- Finally, reduce A and B together to S ($S \rightarrow AB$).



Difference between Top-Down and Bottom-Up Parsing



Key	Top Down Parsing	Bottom Up Parsing
Strategy	starts evaluating the parse tree from the top and move downwards for parsing other nodes. From root -> leaf	starts evaluating the parse tree from the lowest level of the tree and move upwards for parsing the node. From leaf -> root
Starts with	Starts with the start symbol (S).	Starts with the input string. Also called Reduction due to this.
Attempt	attempts to find the left most derivation for a given string.	attempts to reduce the input string to first symbol of the grammar.
Derivation Type	uses leftmost derivation.	uses the rightmost derivation .
Objective	searches for a production rule to be used to construct a string.	searches for a production rule to be used to reduce a string to get a starting symbol of grammar.

Chapter - 4 Syntax Directed Translation

66

- The plain parse tree, the output of the syntax analysis phase, lacks crucial information for a compiler such as;
 - **semantic information:** The parse tree doesn't tell the compiler *how to interpret or evaluate the code* represented by the tree.
 - **semantic rules:** CFG productions themselves don't specify *how to interpret the symbols* they generate. They only define the possible structures. For instance, look at the production rule: $E \rightarrow E + T$
- This production rule simply states that an expression (E) can be formed by another expression (E) followed by a "+" operator and a term (T).
- However, the parse tree generated from this rule wouldn't tell the compiler:
 - what kind of values E and T represent (numbers, strings, etc.) or
 - how the "+" operator should be applied (addition, concatenation, etc.).
 - Hence, another phase of compiler called **semantic analyzer** is introduced.

- **Semantics** of a language provide **meaning** to its constructs, like **tokens** and **syntax structure**.
- **Semantic Analysis** judges whether the syntax structure constructed in the source program derives any **meaning** or **not**.
- That is, the semantic analysis will perform:
 - **Type Checking**: Verifies that variables and expressions are used with compatible data types (e.g., adding integers, not strings).
 - **Variable Scope**: Ensures variables are declared and used within their defined scope to avoid referencing undefined variables.
 - Other Checks: identifying **unused variables**, **checking for potential null pointer exceptions**, and other language-specific rules.
- A **Syntax Directed Translation(SDT)** refers to translation techniques that **integrate semantic actions with parsing** using grammar extensions like Syntax Directed Definitions (SDDs).
 - *They not only define the structure but also attach meaning to the grammar rules.*

- This added meaning allows the compiler to take actions like **type checking** and **code generation** while it parses the code, making it much more powerful than just checking for syntax errors.
- That is;

CFG + semantic rules = Syntax Directed Translations

Why Semantic Analyzer?

- For example: **int a = "value";**
- ✎ **Lexical Analysis:** successfully identify "int," "a," "=", "value," and ";". It wouldn't raise an error.
- ✎ **Syntax Analysis (Parsing):** the statement `int a = "value";` follows a common grammar rule for assignment statements. Therefore, syntax analysis wouldn't raise an error either.
- ✎ **Semantic Analysis:** this phase focuses on the meaning and relationships between the identified tokens. It identifies a type mismatch i.e integer (int) assigned string value "value" causing a semantic error.

- To make sense of the code and perform tasks like **type checking** or **code generation**, **variable scope**, **whether variables are used or not**, the compiler needs additional information beyond the structure of the code.
- To do all this tasks the semantic analysis, uses techniques like:
 - **Attribute Grammars:** These extend CFGs with attributes to define how attribute values are computed during parsing.
 - **Symbol Tables:** These track information about variables, functions, and other entities used in the code.

1. Attribute Grammar

70

- **Attribute grammar** is a special form of CFG where some **additional information (attributes)** are **appended** to one or more of its non-terminals (sometimes on terminals too) in order to provide context-sensitive information.
 - Each attribute has well-defined domain of values, such as *integer, float, string, and expressions*.
- Imagine a grammar rule for an assignment statement: **Stmt** $\rightarrow$ **id** = **Expr** ;
- **Without Attribute Grammars**: the parse tree would only show the structure (id, =, Expr).
 - We wouldn't know if **id** and **Expr** represent compatible types (e.g., both numbers).
- **With Attribute Grammars**: we can attach attributes to symbols:
 - **id.type**: type of the variable (e.g., integer).
 - **Expr.type**: type of the expression result.
- That is, with an attribute grammar we can define semantic rules associated with the production rule:
 - Check if **id.type** and **Expr.type** are compatible (**type checking**).
 - Set **id.value** to the evaluated **value of Expr** (assuming assignment).

- **Example #1:** $E \rightarrow E + T$ $\{ E.value = E.value + T.value \}$
- The right part of the CFG contains the ***semantic rules*** that specify how the grammar should be interpreted. *Here, the values of non-terminals E and T are added together and the result is copied to the non-terminal E .*
- Based on the way the attributes get their values, attribute grammar can be broadly divided into **two** categories :
 - a. Synthesized attribute and*
 - b. Inherited attributes.*

a. Synthesized Attribute

72

- **Synthesized attributes** are computed **bottom-up in the syntax tree** and depend only on the attributes of the **node's children**.
 - They are useful for propagating information up the tree.
- **Example:** Consider the production rule:
$$E \rightarrow E1 + E2 \{ E.val = E1.val + E2.val; \}$$
- In this example, the synthesized attribute "val" of the non-terminal E is computed by adding the values of the synthesized attributes of its children E1 and E2.

- Let us see another example;
- Given the following grammar

G → PQRS

- Here, G is referred to be a synthesized attribute if it receives values from its children nodes which are P, Q, R, and S.

b. Inherited Attribute

74

- **Inherited attributes** are computed **top-down** in the syntax tree and depend on the attributes of the **node's parent** and **siblings**.
- They are useful for **passing information down the tree**.
- **Example 1:**

G → PQRS

- Here, P can receive values from values for G, Q, R, S.
 - Likewise S can receive values from G, P, Q, R.
 - We can also say S can provide values for Q, P, R.
- **Note here;** G is parent and hence S cant provide values for G. But it can receive values from G.

2. Symbol Tables

75

- A **symbol table** is a data structure used in compilers to **store information about identifiers** (variables, functions, etc.) encountered during the parsing phase.
- Each entry in the symbol table typically includes the **identifier's name**, its **type** (integer, string, etc.), its **scope** (local or global variable), **memory location** (for variables), and other relevant information.
- **Example:** Consider a simple program snippet like

```
int x = 5;  
void func(int y)  
{ ... }
```

- In the above code snippet the **symbol table** would have entries for:

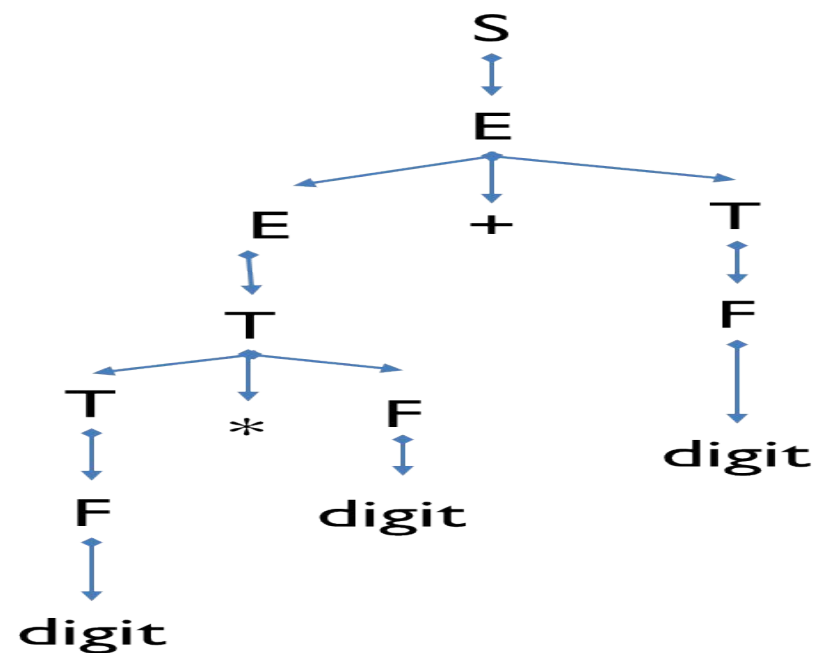
Identifier/ Symbol	Type	Scope	Other properties
X	int	global	—
func	void	global	—
y	int	local	—

- Note: In **synthesized annotations** parents can depend on children, and in an **inherited annotations** children can depend on parents or their siblings.
- A typical parse tree, showing the value(s) of nodes attribute(s) is called an **annotated parse tree**.
- Look at the following(parse tree vs annotated parse tree);

Productions	Semantic rules
$S \rightarrow E n$	$S.val = E.val$
$E \rightarrow E + T$	$E.val = E.val + T.val$
$E \rightarrow T$	$E.val = T.val$
$T \rightarrow T * F$	$T.val = T.val * F.val$
$T \rightarrow F$	$T.val = F.val$
$F \rightarrow (E)$	$F.val = E.val$
$F \rightarrow \text{digit}$	$F.val = \text{digit.lexval}$

SDD

Input: 4 * 5 + 6



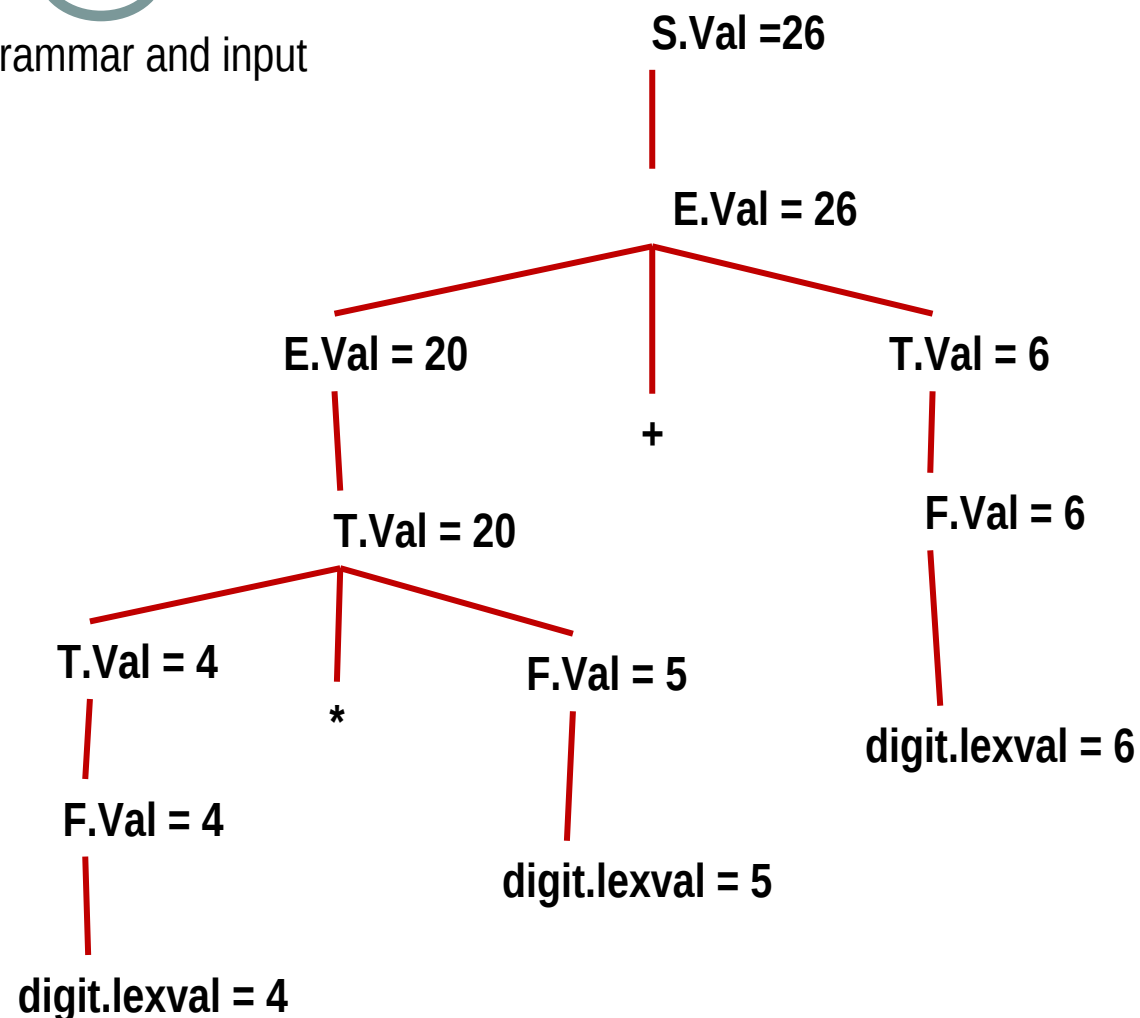
1. Parse tree

- Example:** draw the annotated parse tree for the following grammar and input strings:

 $S \rightarrow E$
 $E \rightarrow E + T \mid T$
 $T \rightarrow T * F \mid F$
 $F \rightarrow (E) \mid \text{digit}$

Input: 4 * 5 + 6

Productions	Semantic rules
$S \rightarrow E$	$S.val = E.val$
$E \rightarrow E + T$	$E.val = E.val + T.val$
$E \rightarrow T$	$E.val = T.val$
$T \rightarrow T * F$	$T.val = T.val * F.val$
$T \rightarrow F$	$T.val = F.val$
$F \rightarrow (E)$	$F.val = E.val$
$F \rightarrow \text{digit}$	$F.val = \text{digit.lexval}$



Dependency Graph

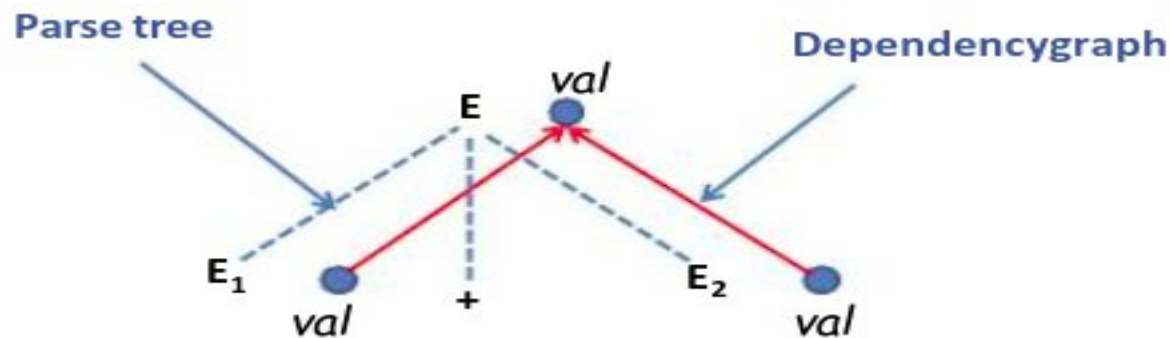
78

- **Dependency graphs** are a useful tool for determining an evaluation order for the attribute instances in a given parse tree.
- They are also used to represent the flow of information among the attributes in a parse tree.
- While an **annotated parse tree** shows the values of attributes, a *dependency graph* helps us determine how those values can be computed.
- In other words, inter dependency between *Synthesized* and **inherited attributes** at node in the parse tree is called *dependency graph*.

- Example: $E \rightarrow E_1 + E_2$ that is;

- Production: $E \rightarrow E_1 + E_2$

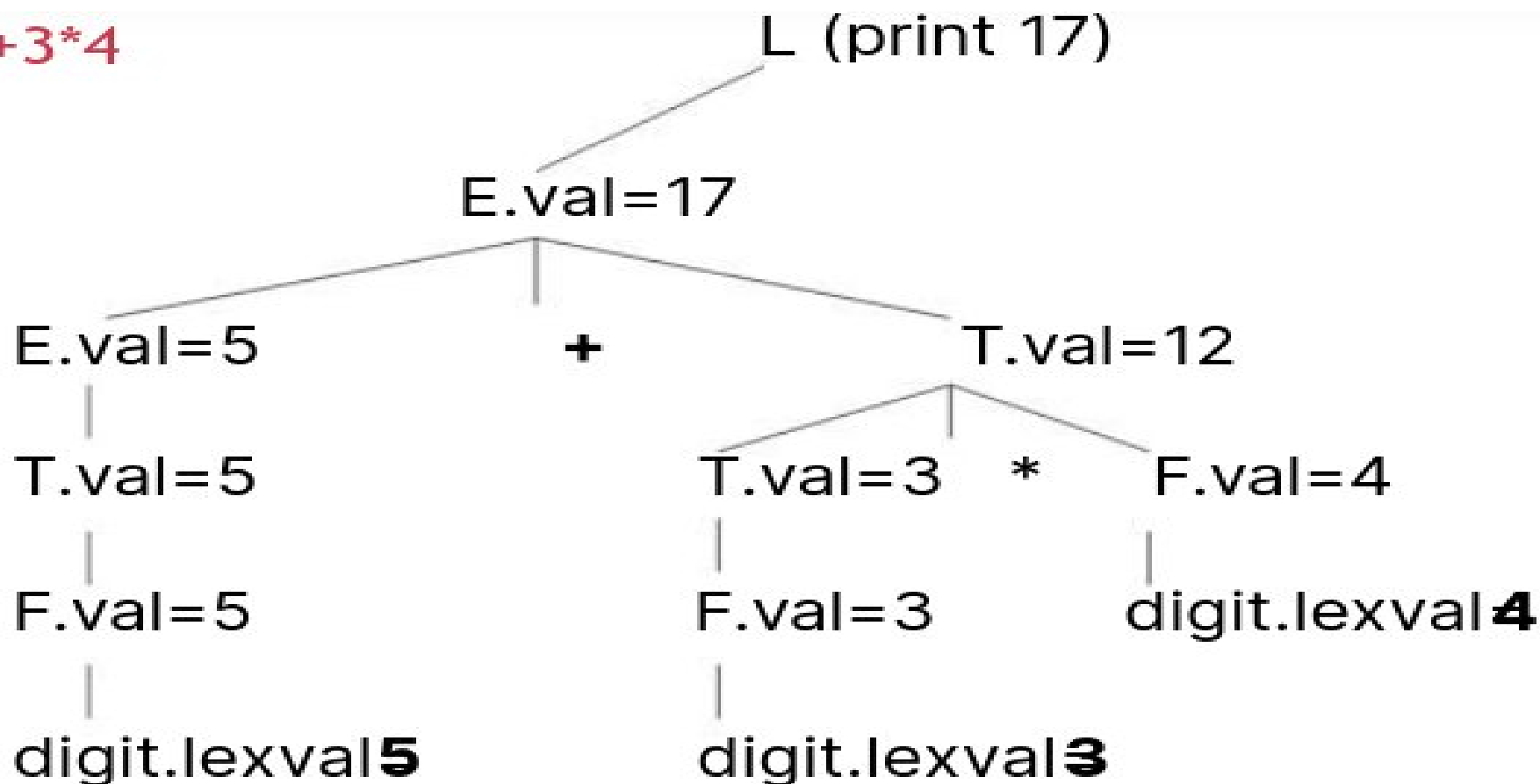
Semantic rules: $E.val = E_1.val + E_2.val$



- *$E.val$ is synthesized from $E_1.val$ and $E_2.val$*
- The edges to $E.val$ from $E_1.val$ and $E_2.val$ shows that $E.val$ depends on $E_1.val$ and $E_2.val$

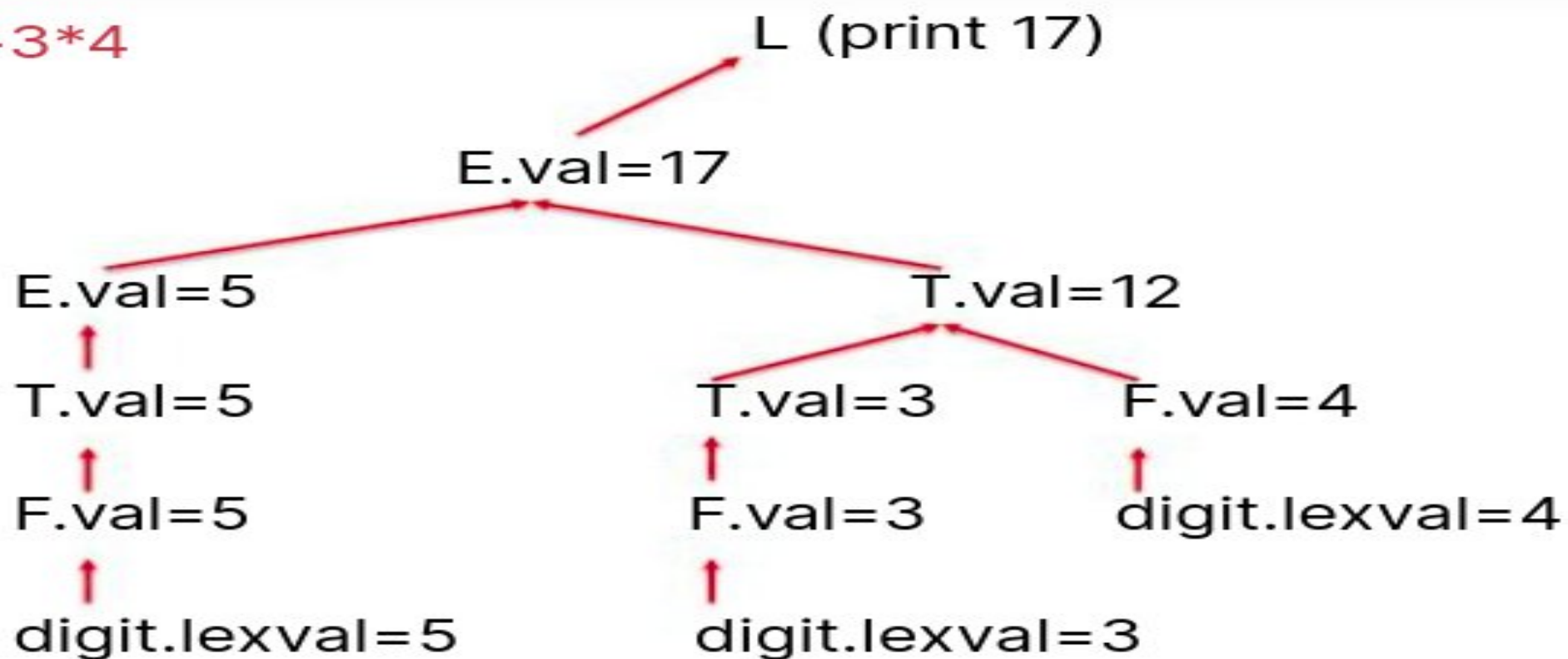
- **Example:** given the following **Annotated Parse Tree**

Input: **5+3*4**



- Example: its **Dependency Graph** would then be

Input: $5+3*4$



Review Question

82

1. *Differentiate lexical analysis, syntax analysis and semantic analysis.*
2. *Why do we need semantic analysis? List any two reasons.*
3. *What is annotated parse tree? What makes it different from syntax tree. Show the difference using tree structure.*

Chapter -5 Type Checking

83

- **Type checking** is a crucial stage in compiler design that ensures the compatibility of data used in a program.
- It verifies that the operations performed on variables or expressions are valid based on their data types (e.g., integer, string, boolean).
- This helps to prevent compile time errors and promotes program correctness.
- Much of what we do in the *semantic analysis phase* is *type checking*.
- **The main goal of type checking is therefor:**
 - check that the source program should follow the **semantic** conventions of the source language.
 - to check whether the source program is maintained correctly or not.
 - to check the **data type assignments** and **type-casting** of the data types.
 - it checks the **type rules** of the language.

- **Type checking** involves comparing the type of an expression with the expected type, and ensuring that they are compatible.
- **For example**, adding two values of different types (such as a string and a number) would result in a **type error**.

```
String message = "Hello"
```

```
int number = 10
```

```
sum = message + number
```

```
# This would result in a type error
```

- Once the type checking is complete, the compiler can generate code that is optimized for the specific types that are used in the program.
 - This can lead to faster and more efficient code. Hence this type checking is an important part of the compiler optimization process.

Compile-time	Runtime
The compile-time errors are the errors which are produced at the compile-time, and they are detected by the compiler.	The runtime errors are the errors which are not generated by the compiler and produce an unpredictable result at the execution time.
In this case, the compiler prevents the code from execution if it detects an error in the program.	In this case, the compiler does not detect the error, so it cannot prevent the code from the execution.
It contains the syntax and semantic errors such as missing semicolon at the end of the statement.	It contains the errors such as division by zero, determining the square root of a negative number.

Steps involved in designing a type checker for a compiler

86

- When designing a type checker for a compiler, we need to:
 - Identify the types that are available in the language(integers, floats, booleans, etc).
 - Identify the language constructs that have types associated with them(identify which elements in the language have associated types i.e variables, expressions, function or return types?).
 - Identify the semantic rules for the language(define a set of rules that govern how different types can be used together).
- A language is considered **strongly- typed** if each and every type error is detected during compilation. This means the type checker enforces strict type rules, and any mismatch between expected and actual types will be flagged as an error before the program even attempts to run.

Rules of Type Checking

87

- Type checking follows specific rules that define valid operations based on data types. These rules vary depending on the programming language.
- Here are some common rules:
 - **Assignment:** The type of the expression on the right side of an assignment must be compatible with the type of the variable on the left side.
 - **Arithmetic Operations:** Operations like addition, subtraction, multiplication, etc., must involve operands of compatible numeric types (e.g., integer + integer).
 - **Comparisons:** Relational operators ($>$, $<$, $==$) must compare operands of the same type (e.g., integer compared to integer).
 - **Function Calls:** The arguments passed to a function must match the expected types of the function's parameters.

Type Checking Preventions

88

- Type checking is a fundamental safeguard in compiler design that ensures the compatibility of data types used in a program. It helps prevent to errors and promotes code correctness. Here's a breakdown of how **type checking addresses several potential issues**:
 - Application of a function to wrong number of arguments.

```
int add(int x, int y) {  
    return x + y;  
}
```

```
int result = add(5);           // Type error: function expects 2 arguments, 1 provided
```


- Application of integer functions to floats.

```
int absoluteValue(int x) {  
    return -x;  
}
```

```
int result = absoluteValue(3.14);    // Potential type error: function expects integer arg
```

- Use of undeclared variables in expressions.

```
int x = 10;  
int result = y + 5;                  // Type error: variable 'y' is undeclared
```

- Array indices out of bounds.

```
int numbers[5] = {1, 2, 3, 4, 5};  
int value = numbers[7];              // Type error: index 7 is out of bounds (valid indices 0 to
```

Types of Type Checking

90

- Based on the **timing of when the type compatibility of data is verified** in a program we can classify the type checking into two main types:
 - 1. Static Type Checking**
 - 2. Dynamic Type Checking**

1. Static Type Checking

91

- **Static Type Checking** happens during **compilation**, without actually executing the program.
- The **compiler analyses** the source code and verifies type compatibility based on the declared types of variables and expressions.
- *Languages like Pascal, C and Java have static type checking.*
- **Example:** Java

```
int x = 10;
int y = x + 5; // valid
string message = "Hello" + x; // static type error:
```

2. Dynamic Type Checking

92

- **Dynamic Type Checking** occurs at **runtime**, while the program is actually running.
- The **interpreter** or **virtual machine** checks type compatibility before performing operations.
- *Common dynamically typed languages are : JavaScript, PhP and Python etc.*
- **Example:** Python

```
x = 10
```

```
y = "Hello"
```

```
print(x + y) # This is dynamic type error:
```

Choosing Type Checking

93

- While static type checking provides benefits for error detection and code clarity, dynamic type checking offers more flexibility in languages like Python.
- The choice between static and dynamic type checking depends on several factors:
 - **Language Design Goals:** If the emphasis is on **strong type safety** and **early error detection**, **static checking** might be preferred.
 - **Target Audience:** Languages designed for **beginners** or **scripting** might favour **dynamic checking** for a more accessible learning curve.
 - **Development Style:** Some developers prefer the discipline and predictability of static type systems.

Type Conversion

94

- Sometimes, data needs to be transformed from one type to another during program execution. This process is called **type conversion**.
- There are two main types: **Implicit** and **Explicit** type conversions.

1. Implicit Conversion (Coercion)

- The conversion from one type to another type is known as **implicit** if it is to be **done automatically by the compiler**.
- Implicit type conversions are also called **Coercion** and coercion is limited in many languages.
- **Example:** as in java

```
int x = 10;  
double y = x; // Implicit conversion
```

2. Explicit Conversion (Casting):

- The **programmer instructs the compiler to convert data** from one type to another using casting operators (e.g., (int), (double), etc.).
- This provides more control and clarity about the intended behavior.
- **Example:** as in java

```
double pi = 3.14159;
```

```
int radius = (int)pi; // Explicit conversion from double to int (truncates decimal)
```

Key Differences:

Feature	Implicit Conversion (Coercion)	Explicit Conversion (Casting)
Initiated by	Compiler	Programmer
Control	Less control for programmer	More control for programmer
Error handling	May lead to unexpected results	Potential errors can be flagged
Example	<code>int x = 1.5;</code> (may lose precision)	<code>int x = (int)1.5;</code> (explicit truncation)

Compiler tools (Note them)



- **Lexical Analysis tools:** Lex, FLex, JFLex
- **Syntax Analysis tools:** Bison (GNU), Yacc, ANTLR

Any
Question?

A hand holding a pen, pointing at the text 'Any Question?'. The hand is positioned at the bottom right of the text, with the pen tip touching the end of the word 'Question?'. The background is white with faint, repeating watermarks of the word 'QUESTION' and a camera icon.

Review Questions(1)



1. What is the output of the lexical analyzer for `if (x > 5) { y = 1; }`?
 - a) **Tokens: (if), ((), (id,x), (>,5), (), ({}), (id,y), (=,1), (;), ({}))**
 - b) Abstract syntax tree
 - c) Machine code
 - d) Symbol table entries
2. Which phase detects the error in `int x = "string";`?
 - a) Lexical
 - b) Syntax
 - c) **Semantic**
 - d) Code Generation
3. Which tool generates parsers from CFG specifications?
 - a) Lex
 - b) **Yacc/Bison**
 - c) GCC
 - d) GDB
4. What does the semantic analyzer check in `float x = 10;`?
 - a) **Implicit type conversion**
 - b) Missing semicolon
 - c) Undeclared variable
 - d) Operator precedence

Review Questions(2)



5. What is the output of syntax analysis for $x = y + z$;

- a) Tokens
- b) Parse tree/AST**
- c) Symbol table
- d) Three-address code

6. Which error does `int x = y + "5";` trigger?

- a) Lexical
- c) Type mismatch**
- b) Syntax
- d) Runtime

7. Which phase generates assembly code?

- a) Lexical Analysis
- b) Target code generation**
- c) Semantic Analysis
- d) Intermediate Code Generation

8. Which grammar rule is left-recursive?

- a) $A \rightarrow \epsilon$
- b) $A \rightarrow aA \mid b$
- c) $A \rightarrow a \mid b$
- d) $A \rightarrow Aa \mid b$**

Review Questions(3)



9. Which attribute type is passed from parent to child nodes in SDT?

- a) Dynamic
- b) Synthesized
- c) Static
- d) Inherited**

10. How does operator precedence parsing resolve ambiguities?

- a) By random selection
- b) By backtracking
- c) By defining precedence/associativity rules**
- d) By skipping errors

11. What is the role of the semantic stack in SDT?

- a) Optimize code
- b) Track scopes
- c) Allocate memory
- d) Store attribute values during parsing**

12. What distinguishes static from dynamic type checking?

- a) Static checks types at compile-time**
- b) Dynamic is faster
- c) Static allows implicit conversions
- d) Dynamic requires no symbol table

Review Questions(4)



13. What is the first phase of compilation?

a) Semantic Analysis

b) Syntax Analysis

c) Lexical Analysis

d) Code Generation

14. What does a lexical analyzer remove?

a) Syntax errors

b) Comments/whitespace

c) Type errors

d) Static types

15. Which grammar is used for syntax analysis?

a) Context-Free Grammar

b) Regular Grammar

c) Context-Sensitive Grammar

d) Unrestricted Grammar

16. What is the final phase of compilation?

a) Optimization

b) Code Generation

c) Linking

d) Loading

Review Questions(5)



17. What is the purpose of a syntax tree?

- a) Allocate memory
- b) Store tokens
- c) **Represent hierarchical structure of source code**
- d) Optimize loops

18. Which phase detects missing semicolons?

- a) **Syntax Analysis**
- b) Lexical Analysis
- c) Semantic Analysis
- d) Code Generation

19. What is the role of the loader?

- a) Optimize IR
- b) Generate machine code
- c) Resolve symbols
- d) **Load executable into memory**

20. Which phase performs type checking?

- a) Code Generation
- b) Lexical Analysis
- c) Syntax Analysis
- d) **Semantic Analysis**

Review Questions(6)



21. What is the primary function of a preprocessor?

- a) **Handle macros and file inclusion**
- b) Generate assembly
- c) Optimize code
- d) Allocate registers

22. Which parsing technique cannot handle left recursion?

- a) LR(1)
- b) **Recursive Descent**
- c) LALR(1)
- d) SLR(1)

23. What is the primary purpose of a dependency graph?

- a) **Determine evaluation order of attributes**
- b) Allocate memory
- c) Tokenize input
- d) Resolve conflicts

24. What is the primary role of the assembler?

- a) Resolve symbols
- b) Optimize IR
- c) **Convert assembly to machine code**
- d) Tokenize input

Review Questions(7)



25. Which parsing technique uses backtracking?

- a) LALR(1)
- b) LR(1)
- c) **Recursive Descent**
- d) SLR(1)

26. What is the primary advantage of separating lexical and syntax analysis?

- a) Reduced memory usage
- b) Faster compilation
- c) **Modularity and simplified error handling**
- d) reduce time consumption

27. What is the primary purpose of syntax-directed translation?

- a) Allocate registers
- b) Generate tokens
- c) **Associate semantic rules with grammar productions**
- d) Optimize code generation

28. Which attribute is computed bottom-up in SDT?

- a) Inherited
- b) **Synthesized**
- c) Static
- d) Dynamic

Review Questions(8)



29. What does static type checking enforce?

a) Variable naming conventions

b) Array bounds at runtime

c) Memory allocation

d) Type compatibility at compile-time

30. What is the output of the lexical analyzer?

a) Parse tree

b) Tokens

c) Machine code

d) Symbol table

31. What is the purpose of a symbol table?

a) To store tokens

b) To store information about identifiers

c) To generate machine code

d) To optimize intermediate code

32. Which of the following is NOT a lexical error?

a) Misspelled keyword

b) Invalid character in an identifier

c) Division by zero

d) Unmatched string literal